

Folien-Erklärung — jede Zeile verständlich (Vortragsbegleiter)

Zweck: Zu jeder Folie steht hier, was jede Zeile **bedeutet**, **warum** sie dort steht und (wo nützlich) welche **Nachfrage** kommen kann und wie du antwortest. Ziel: Du kannst die Folien nicht nur vorlesen, sondern **frei erklären**.

Lies zusätzlich: folien_sprechzettel.pdf = die Anker + ein gesprochener Satz je Folie. Dieses Dokument hier ist das Verständnis, der Sprechzettel die Erinnerungsstütze.

Die ganze Arbeit in 3 Sätzen (roter Faden)

1. **Frage:** Erklärt die **Netznähe** (wie weit ein Server weg ist) die Latenzunterschiede zwischen Cloud-AI-Diensten — oder steckt der Unterschied im **Backend** (Hardware + Modell)?
2. **Methode:** Ich messe in **drei Schichten** (Netz / Paketebene / API), damit ich Netz-Anteil und Backend-Anteil **getrennt** sehe — und ich eiche meine Timer gegen die echten Netzwerk-Pakete.
3. **Befund:** Drei LLM-Anbieter hängen am **selben** Cloudflare-Knoten in Frankfurt (~1 ms), antworten aber bis zu **~7,3× unterschiedlich** schnell. Gleiches Netz, riesiger Unterschied → **die Netznähe erklärt die Spreizung nicht**. (Wasserdicht ist diese *negative* Aussage.)

Wichtigste Vokabeln vorab:

- **RTT** (Round-Trip-Time): Zeit für ein Signal hin **und** zurück zum Server. Maß für „wie weit weg“.
- **Cold-Start:** Jede Messung baut eine *frische* Verbindung auf (kein Wiederverwenden) — wie ein Nutzer, der gerade erst zu sprechen beginnt.
- **ttft / ttfa / ttfp:** Time-to-first-**T**oken (LLM) / -first-**A**udio (TTS) / -first-**P**artial (STT) — also „wie lange bis das **erste** Stück Antwort kommt“.
- **Edge / Backend:** *Edge* = der nächstgelegene CDN-Knoten (Vorposten, nimmt die Verbindung an). *Backend* = der eigentliche Rechen-Server, der das KI-Modell ausführt (kann weit weg sein).
- **A4 / Median der Slot-Mediane:** Aggregationsregel — erst Median je Zeit-Slot, dann Median über die Slots. Robust gegen einzelne Ausreißer-Slots.

Folie 1 — Titelfolie

Was draufsteht: Der Arbeitstitel als Frage „*Erklärt Netznähe die Latenzunterschiede?*“, der Untertitel „*Eine drei-schichtige Messstudie zu Cloud-AI-Sprachdiensten (STT · LLM · TTS) aus EU-Sicht*“, und ein Badge „**Konsultation · Setup-Stand**“.

- **Titel als Frage** — bewusst **offen** formuliert. Eine Frage kann man nicht „überziehen“: Egal wie das Ergebnis ausfällt, der Titel bleibt korrekt. (Ein Behauptungs-Titel wie „Backend schlägt Geografie“ wäre angreifbar wegen des Modellgrößen-Confounds — dazu Folie 17.)
- **STT · LLM · TTS** — die drei Bausteine einer Sprach-Pipeline: **S**peech-**t**o-**T**ext (Spracherkennung), **L**arge **L**anguage **M**odel (Antwort generieren), **T**ext-**t**o-**S**peech (Sprachausgabe).
- **„aus EU-Sicht“** — gemessen wird von einem Server in Europa (Frankfurt); das ist die Perspektive der Forschungsfrage.
- **Badge „Setup-Stand“** — signalisiert: heute geht es v.a. um **Setup & Methodik**, nicht um die fertige Ergebnis-Interpretation.

Ein Satz: „*Meine Leitfrage ist bewusst als offene Frage formuliert, damit ich nicht mehr behaupte, als die Daten hergeben.*“

Folie 2 — Agenda

Was draufsteht: Vier nummerierte Blöcke — **1 Setup** (hervorgehoben, „Schwerpunkt“), **2 Methodik**, **3 Analyse**, **4 Abschluss**.

- **Setup als Schwerpunkt** — du betonst zuerst das Fundament (womit/wie gemessen wird), weil genau das beim letzten Mal gefehlt hat. Erst Vertrauen ins Setup, dann die Zahlen.
- **Reihenfolge Setup → Methodik → Analyse** — bewusst „Methode vor Zahlen“: Der Prüfer soll erst sehen, dass sauber gemessen wird, bevor Ergebnisse kommen.

Ein Satz: „Der Schwerpunkt liegt heute auf dem Setup, weil das Fundament zuerst stehen muss.“

Folie 3 — Forschungsfrage

- **„Pipeline: STT → LLM → TTS (sequenziell)“** — die drei Dienste laufen **nacheinander**: erst wird gesprochen→Text, dann Text→Antwort, dann Antwort→Sprache. Jede Stufe wartet auf die vorige. Deshalb addieren sich die Latenzen.
- **„Latenz entscheidet die Nutzbarkeit“** — bei einem Sprachassistenten merkt der Nutzer jede Verzögerung; Latenz ist also kein Nebenaspekt, sondern *das* Qualitätskriterium.
- **„Kernfrage: Netz/Infrastruktur vs. Backend?“** — die eigentliche Frage: Kommt der Latenzunterschied vom **Netzweg** (Distanz, Route) oder von der **Verarbeitung** im Server (Hardware, Modellgröße)?
- **„Faktoren: RTT, Protokoll, Hosting-Region“** — die *Netz-Seite*, die man üblicherweise verdächtigt: Entfernung (RTT), Übertragungsart (WebSocket/HTTPS), Standort des Rechenzentrums.
- **„Bewusst offen — auch ein Nein zählt“** — wenn herauskommt „das Netz erklärt **wenig**“, ist das ein **gültiges, sogar starkes** Ergebnis. Wissenschaftlich sauber, weil ergebnisoffen.
- **„Zweite Achse: Verfügbarkeit (separat)“** — neben der Geschwindigkeit messe ich **Zuverlässigkeit** (wie oft scheitert ein Aufruf). Schnell **und** unzuverlässig ist nicht „gut“ — deshalb getrennte Achse.

Mögliche Nachfrage: „Warum trennen Sie Latenz und Verfügbarkeit?“ → „Sonst gewinnt ein Anbieter scheinbar, weil seine langsamen/gescheiterten Aufrufe aus dem Median herausfallen (Survivorship-Bias).“

Folie 4 — Abschnitt 1: Setup (Trenner)

Reiner Abschnitts-Trenner. Unterpunkte kündigen die drei Setup-Folien an: **Vantage Point**, **Provider-Matrix**, **Messdesign**.

Folie 5 — Vantage Point

Vantage Point = der feste **Mess-Standort**, von dem aus alle Messungen laufen.

- **„AWS EC2 eu-central-1 (Frankfurt)“** — ein Server in der Amazon-Cloud, Region Frankfurt. EU-Standort, weil die Forschungsfrage die EU-Perspektive ist; Frankfurt ist ein zentraler EU-Internetknoten.
- **„c6i.large — bewusst NICHT burstable“** — der gewählte Server-Typ hat **garantierte** CPU-Leistung.
- **„Burstable würde drosseln → Latenz-Sprünge ohne Netz-Ursache“** — billige „burstable“-Typen (t2/t3) laufen auf CPU-Guthaben; ist das aufgebraucht, **drosselt** Amazon die CPU. Dann springen meine Millisekunden-Timer — **ohne** dass das Netz schuld ist. Das würde die Messung verfälschen, also vermieden.
- **„CPU-Steal-Time je Slot geloggt“** — *Steal-Time* = wie viel CPU mir der Hypervisor „wegnimmt“. Ich protokolliere sie pro Messrunde und kann so **belegen**, dass keine Drosselung stattfand — nicht nur

behaupten.

- „**IPv4 durchgängig erzwungen**“ — alle Schichten messen über dieselbe Adressfamilie (IPv4). Sonst könnte Schicht 1 eine andere IP (IPv6) messen als der echte Mess-Request (IPv4) → Äpfel/Birnen. Erzwungen = Konsistenz.
- „**OpenSSL 3.x (LibreSSL meldete TLS-Zeiten falsch)**“ — auf dem Mess-Server läuft die korrekte Verschlüsselungs-Bibliothek; mein Entwicklungs-Mac meldete TLS-Versionen falsch. Deshalb gelten nur die EC2-Werte.
- „**Limitation: ein Vantage Point — Kernbefund unberührt**“ — **Ehrliche Grenze:** Ich messe nur von Frankfurt aus, die RTT-/Edge-Zahlen sind also FRA-spezifisch. **Aber:** Der Kernbefund (Folie 16) gilt trotzdem, weil er bei *konstanter* Netz-Distanz auftritt (alle drei LLM ~1 ms) — er hängt nicht am Standort.

***Mögliche Nachfrage:** „Nur ein Standort — ist das nicht zu wenig?“ → „Für die RTT-Zahlen ja, das nenne ich als Limitation. Der Kernbefund ist davon unabhängig, weil er bei gleicher Netzdistanz für alle drei LLM gemessen ist.“*

Folie 6 — Provider-Matrix · 9 Endpunkte

- „**9 Endpunkte über 6 Anbieter × 3 Kategorien**“ — ich messe **9 Mess-Ziele:** je drei für STT, LLM, TTS. Es sind **6 Firmen** (Deepgram, Rev.ai, Azure, OpenAI, Groq, Mistral) — manche stellen mehrere Endpunkte (z.B. Deepgram STT *und* TTS). Wichtig: „Endpunkt“ ≠ „Anbieter“ (das fällt einem Mess-Prüfer auf).
- „**Feste Inputs je Kategorie**“ — jede Kategorie bekommt **denselben** Eingabe-Reiz für alle Anbieter (gleiche Audiodatei für STT, gleicher Prompt für LLM, gleicher Satz für TTS) → fairer Vergleich.
- „**Fairer Cross-Provider-Vergleich**“ — nur mit identischem Input ist der Latenzvergleich zwischen Anbietern aussagekräftig.

Zur Tabelle (Kategorie | Anbieter | Modell | Region | Protokoll):

- **Region (deklariert)** = was der Anbieter **angibt** (USA, Italien, EU/Frankreich). „Deklariert“, weil das nicht heißt, dass die Verbindung dort *endet* (s. Edge/Host, Folie 11).
- **Protokoll:** STT nutzt **WebSocket** (dauerhafte Zwei-Wege-Verbindung — nötig, um Audio live zu streamen). LLM/TTS nutzen **HTTPS + SSE / Streaming** (Antwort kommt häppchenweise zurück).
- **Groq „LPU“, OpenAI „GPU“** — unterschiedliche Inferenz-Hardware (LPU = spezieller Chip für Sprachmodelle; GPU = Grafikkarte). Wichtig später für „Backend statt Geografie“.

***Mögliche Nachfrage:** „Warum diese Anbieter?“ → „Pro Kategorie ein Mix aus US- und EU-Hosting und verschiedenen Protokollen; Begründung steht in der Anbieter-Auswahl der Methodik.“*

Folie 7 — Messdesign · Cold-Start · interleaved

- „**Cold-Start: frische TCP+TLS je Call**“ — jeder einzelne Messaufruf baut die Verbindung **neu** auf (TCP = Verbindungsaufbau, TLS = Verschlüsselungs-Handschlag). Kein Recyclen alter Verbindungen.
- „**Kein Pooling · kein Keep-Alive**“ — *Connection-Pooling/Keep-Alive* = Verbindungen offenhalten und wiederverwenden. Mache ich **bewusst nicht**, weil ich den realen Start einer neuen Sprach-Sitzung messe — inklusive Verbindungsaufbau, der aus EU-Sicht gerade der interessante Teil ist.
- „**Feste Inputs je Kategorie**“ — s. Folie 6 (gleicher Reiz für alle).
- „**STT 5 s-WAV · LLM Capital-Prompt · TTS Gruß**“ — konkret: eine ~5-Sekunden-Audiodatei für STT; für LLM die Frage nach der Hauptstadt Deutschlands; für TTS ein kurzer Begrüßungssatz.
- „**Interleaved Round-Robin, 100 Runden**“ — *interleaved* = ich messe **abwechselnd** reihum alle Anbieter, 100 Runden lang, statt erst 100× Anbieter A, dann 100× B. So sehen alle Anbieter **dieselbe**

zeitliche Verteilung der Netzbedingungen → fair.

- „**Startreihenfolge rotiert**“ — in jeder Runde startet ein anderer Anbieter zuerst, damit keiner systematisch bevorzugt/benachteiligt wird.
- „**Fehlschlag blockiert die Runde nicht**“ — scheitert ein Aufruf, läuft die Runde weiter; der Fehler wird **gezählt** (für die Verfügbarkeits-Achse), nicht verschluckt.

Mögliche Nachfrage: „Warum kein Connection-Pooling?“ → „Ich will den Kaltstart einer neuen Sitzung messen; Warm-Reuse würde gerade den Verbindungsaufbau verstecken.“

Folie 8 — Abschnitt 2: Methodik (Trenner)

Trenner. Kündigt an: 3 Schichten · RTT · Edge/Host · Paket-Eichung · Metrik-Uhr · Kampagne. Inklusive der zwei kritischen Erklärschritte (Folie 11 & 12).

Folie 9 — Drei-Schichten-Architektur

Die Grafik zeigt drei gestapelte Ebenen (Layer 1 unten bis Layer 3 oben).

- „**Die Frage liefert die Zerlegung**“ — die Forschungsfrage (Netz vs. Backend) **ist** der Grund für die drei Schichten: jede Schicht trennt einen Anteil ab.
- „**Layer 1 → Host · Layer 3 → volle URL**“ — Layer 1 misst die Zeit **bis zum Host** (Server-Eingang). Layer 3 misst über die **volle Adresse**, also inklusive der Verarbeitung im Inneren.
- „**Differenz = Backend (Hardware + Modell)**“ — was Layer 3 **mehr** braucht als Layer 1, ist die reine **Verarbeitungszeit** (das Backend). Das ist der Trick, um Netz von Rechenzeit zu trennen.
- „**Layer 2 macht die Leitung prüfbar**“ — Layer 2 schneidet die echten Netzwerk-Pakete mit und erlaubt, meine Software-Timer gegen die **physische Leitung** zu prüfen (Folie 12).

Was die drei Bänder sagen:

- **Layer 1 (Infrastruktur)**: DNS (Namensauflösung), RTT/Ping, TLS, Traceroute, ASN → misst Netznähe zum Host.
- **Layer 2 (Paketebene)**: zeichnet den Paketverkehr auf → **eicht** den Connect-Timer.
- **Layer 3 (API-Latenz)**: Cold-Start mit ttfp/ttft/ttfa → misst über die **volle URL** (Engine).

Ein Satz: „Layer 1 misst zum Server-Rand, Layer 3 misst durch bis zur Antwort — die Differenz ist die Backend-Verarbeitung.“

Folie 10 — Layer 1 · RTT via TCP-Handshake

- „**TCP-Ping (SYN→SYN-ACK :443) = primär**“ — ich messe die RTT, indem ich den **ersten Schritt des Verbindungsaufbaus** stoppe: Mein Rechner schickt ein SYN (Verbindungswunsch) an Port 443 (HTTPS), der Server antwortet mit SYN-ACK. Die Zeit dazwischen = **eine** RTT.
- „**Vergleichbar bei allen 9 (ICMP teils geblockt)**“ — der klassische Ping (ICMP) wird von einigen CDN-Servern blockiert, funktioniert also nicht überall. TCP auf Port 443 muss **jeder** API-Server annehmen → eine Messgröße, die bei allen 9 gleich funktioniert.
- „**Trifft den echten API-Port**“ — ich messe genau den Port, über den später auch die echten Daten laufen (443), nicht einen Nebenkanal, den manche Anbieter anders behandeln.
- „**Deckt sich mit dem Connect-Timer aus Layer 3**“ — dieselbe Größe (eine RTT) taucht in Layer 3 als Teil des Verbindungsaufbaus wieder auf → die Schichten sprechen dieselbe Sprache, die Querverbindung ist sauber.
- „**Validierung: TCP-Ping ≈ ICMP-Ping**“ — dort, wo **beide** funktionieren, stimmen sie überein. Damit beweise ich, dass TCP-Ping ein gültiger RTT-Ersatz ist (Selbst-Validierung der Methode).

Mögliche Nachfrage: „Warum nicht einfach ICMP-Ping?“ → „Weil ICMP bei mehreren CDN-Endpunkten blockiert ist; TCP:443 geht bei allen neun und entspricht genau dem, was Layer 3 als ersten RTT misst.“

Folie 11 — Layer 1 · Edge oder Host? ■ (Antwort auf Einwand 1)

Die Box oben (Leitfrage): „Drei Endpunkte antworten aus Frankfurt mit ~1 ms RTT — woher?“ — Das war der offene Punkt: Wie können US-Anbieter aus Frankfurt nur ~1 ms entfernt sein?

- **„Erklärung: Cloudflare-Edge FRA, nicht Backend“** — die Verbindung endet an einem **Cloudflare-Vorposten (Edge)** in Frankfurt, **nicht** am eigentlichen Rechen-Server. Cloudflare ist ein CDN (Content Delivery Network) mit Knoten weltweit.
- **„~1 ms = reale Edge-Eigenschaft, kein Fehler“** — die niedrige RTT ist **echt** (der Edge *ist* nah), aber sie sagt **nichts** über die Distanz zum echten Backend. Wichtig: kein Messfehler, sondern eine Eigenschaft der Infrastruktur.
- **„Edge nur, wenn alle drei zutreffen:“** — ich behaupte „Edge“ nicht leichtfertig, sondern über einen **Klassifikator mit drei Bedingungen** (alle drei müssen erfüllt sein):
 - **(a) einstellige ms aus FRA** — die gemessene RTT liegt im 1-ms-Bereich.
 - **(b) Ziel-IP in AS13335 (CDN)** — die IP-Adresse gehört zu Cloudflare. AS13335 ist die „Autonomous-System-Nummer“ von Cloudflare (eine Art Netzbetreiber-Kennung). CDN = Content Delivery Network.
 - **(c) Traceroute endet im CDN-AS** — die Routenverfolgung zeigt, dass die Verbindung **im** Cloudflare-Netz endet, nicht durchgereicht wird zu einem fernen Server.

Zur Tabelle (Terminierung | Endpunkte | RTT | Beleg):

- **EDGE (4/9): OpenAI · Groq · Mistral (LLM) + OpenAI-TTS** — ~1 ms, AS13335. Diese vier enden am Cloudflare-Edge.
- **HOST (5/9): Deepgram ~139 ms (zwei US-Carrier AS6461/AS174), Rev.ai ~140 ms (AWS), Azure ~11 ms (AS8075, echtes EU-Rechenzentrum).** Diese fünf enden am **echten** Server.
- **Fußnote: „Edge nur bei 4 von 9 · niedrige RTT bedeutet nicht Edge (Azure ~11 ms ist ein echter Host)“** — zwei Ehrlichkeits-Punkte: (1) Der Edge-Befund gilt **nur für 4 von 9**, nicht fürs ganze Set. (2) Niedrige RTT heißt **nicht automatisch** Edge: Azure ist mit ~11 ms auch nah, aber das ist sein echtes EU-Rechenzentrum (kein CDN-AS → Bedingung b scheitert). Das macht den Befund verteidigbar.

***Ein Satz:** „Die niedrigen RTTs sind kein Rätsel und kein Fehler — drei Anbieter enden an einem Cloudflare-Edge in Frankfurt; das ist über RTT, ASN und Route belegt und gilt nur für vier der neun Endpunkte.“*

Folie 12 — Layer 2 · Paket-Eichung des Connect-Timers ■ (Antwort auf Einwand 2)

Die Box oben: „Validierung: App-Timer unabhängig gegen die Paketebene (PCAP) geeicht.“ — Antwort auf „ich vertraue den Daten nicht“: Ich prüfe meine eigenen Software-Timer gegen die echte Leitung.

- **„30 Cold-Starts je Provider (paketvalidiert: 30/30 bzw. 28/30)“** — ich habe je ~30 Verbindungen aufgezeichnet; bei Azure stimmten 30 von 30, bei Deepgram 28 von 30 (2 gingen am Rand des Aufzeichnungs-fensters verloren). Eine **Verteilung**, kein Einzelwert.
- **„App-Timer vs. SYN→SYN-ACK (PCAP)“** — ich vergleiche die Zeit, die **meine Software** misst, mit der Zeit, die aus den **echten Paketen auf der Leitung** (PCAP = Paketmitschnitt) hervorgeht.
- **„Geeicht: nur der Connect-Timer“** — **ehrliche Reichweite:** Validiert ist **nur** der Verbindungs-Timer (TCP-Handshake), nicht alles.

- „**ttft/ttfa: gleiche Uhr, aber nicht direkt paket-geeicht**“ — die Antwort-Timer (ttft/ttfa) benutzen **denselben** Mess-Mechanismus (dieselbe interne Uhr), starten aber erst beim Absenden des Requests im HTTP-Stack und sind deshalb **nicht direkt** gegen Pakete geprüft. Ich verkaufe also nicht „alle Zahlen paket-validiert“.
- „**Layer 2 = Konsistenz-Check, kein 2. Beweis**“ — Layer 2 und Layer 3 messen **dasselbe** Phänomen (den Handshake), nur auf verschiedenen Ebenen (Kernel-Paketzeit vs. App-Timer). Das ist ein **Plausibilitäts-/Konsistenz-Check**, kein zweiter unabhängiger Beweis.

Zur Tabelle: Azure App-Timer 11,29 ms / SYN→SYN-ACK 11,18 ms / $\Delta +0,11$ ms; Deepgram 139,01 / 138,87 / $\Delta +0,12$ ms.

- **Tagline: „Connect-Timer trifft die Latenz auf der Leitung auf ~0,1 ms genau (...erster Messpunkt als Outlier entfernt)“** — Ergebnis: An beiden Enden der RTT-Skala (11 ms und 139 ms) weicht mein Timer nur ~0,1 ms von der echten Leitung ab. Der kleine konstante Versatz (+0,11 ms) ist die Zeit, die der Kernel nach dem Paket braucht; der allererste Messpunkt (Cold-Cache-Ausreißer) ist ausgeschlossen.

Ein Satz: „Mein Verbindungs-Timer ist gegen die echten Pakete auf ein Zehntel einer Millisekunde genau — das ist die direkte Antwort auf ‚ich vertraue den Daten nicht‘, ehrlich begrenzt auf den Connect-Anteil.“

Folie 13 — Metrik-Asymmetrie · ab wann läuft die Uhr?

Diese Folie räumt eine frühere Verwirrung aus: Bei den drei Kategorien startet die „Stoppuhr“ an **unterschiedlichen** Punkten — das muss man offenlegen, sonst vergleicht man Ungleiches.

- „**Klar deklariert: ab wann die Uhr läuft**“ — ich lege explizit fest, wann gezählt wird.
- „**STT ttfp: ohne Connect (1×-Realtime)**“ — bei Spracherkennung startet die Uhr **erst beim ersten Audio-Chunk** (der Verbindungsaufbau ist abgezogen = *connect-exklusiv*). *ttfp* = Zeit bis zum ersten **Teil-Transkript** (erstes erkanntes Wort). *1×-Realtime* = Audio wird im echten Sprechtempo gestreamt (nicht auf einen Schlag), weil nur so alle Anbieter Zwischenergebnisse liefern.
- „**LLM/TTS ttft/ttfa: inklusive Connect**“ — bei LLM und TTS startet die Uhr **beim Absenden des Requests**, der Verbindungsaufbau ist also **enthalten** (*connect-inklusive*).
- „**E2E zählt Connect nie doppelt**“ — bei der Gesamt-Pipeline (E2E = End-to-End) achte ich darauf, den Verbindungsaufbau **nicht mehrfach** zu addieren.

Zur Tabelle (Kategorie | Primärmetrik | Uhr startet | Connect): STT = ttfp, ab erstem Audio-Chunk, ohne Connect; LLM = ttft, ab Request, inklusive; TTS = ttfa, ab Request, inklusive.

- **Fußnote:** „In die E2E-Pipeline geht das STT-Final (Stream-Ende), nicht ttfp; ttfp unterschätzt die STT-Phase um ~3,8 s.“ — Wichtige Feinheit: Für die **Gesamt-Pipeline** zählt nicht das erste Wort (ttfp), sondern das **fertige** Transkript (das LLM kann erst auf dem **vollständigen** Text loslegen). ttfp wäre als Pipeline-Eingang ~3,8 s zu optimistisch. ttfp ist also gut für „reagiert die Engine zügig“, aber **nicht** die Pipeline-Eingangsgröße.

Mögliche Nachfrage: „Warum für STT eine andere Metrik?“ → „Weil STT-‚Fertig‘-Zeiten von der Stille-/Segmenterkennung des Anbieters abhängen, nicht nur von der Engine. Das erste Live-Wort (ttfp) ist fairer; fürs E2E nehme ich aber das finale Transkript.“

Folie 14 — Kampagne & Aggregation

Linke Spalte (Kampagne = wie viel/wann gemessen wird):

- „**7 Tage × 8 UTC-Slots × n=100, interleaved**“ — über eine Woche, 8 feste Zeitpunkte pro Tag (alle 3 h, in UTC), je 100 Messungen pro Endpunkt. *UTC* = Weltzeit ohne Sommerzeit-Sprünge → reproduzierbar.

- „= 56 Slots = 50.400 Calls" — $7 \times 8 = 56$ Zeit-Slots; mal 100 Messungen mal 9 Endpunkte ≈ 50.400 Aufrufe.
- „UTC-verankert: US-Hoch + -Tief" — die Slots decken sowohl die US-Stoßzeit (Backend stärker belastet) als auch die US-Nacht ab — wichtig, weil viele Backends in den USA stehen.
- „Erhoben & ausgewertet: 56 von 56 Slots (A4)" — Vollkampagne, abgeschlossen (56/56): alle 56 geplanten Slots sind gemessen und mit Regel A4 ausgewertet.

Rechte Spalte (Aggregation = wie aus vielen Zahlen eine wird):

- „**Headline = Median der Slot-Mediane**" — die berichtete Zahl entsteht **zweistufig**: erst der Median je Zeit-Slot, dann der Median über die Slots. So zählt jede Tageszeit gleich viel (robust gegen einzelne schlechte Slots). *Median* = der mittlere Wert (unempfindlich gegen Ausreißer, anders als der Durchschnitt).
- „**Jede Zahl: Bootstrap-95%-CI**" — jeder Wert bekommt ein **Konfidenzintervall** (Unsicherheitsspanne). *Bootstrap* = man zieht viele Stichproben aus den Daten und schaut, wie stark die Zahl schwankt — geeignet für schiefe Verteilungen. (CI = Confidence Interval.) *Steht für die finale Auswertung noch aus.*
- „**„schneller' getestet mit Mann-Whitney / Bootstrap**" — „Anbieter X ist schneller als Y" belege ich mit einem **statistischen Test** (Mann-Whitney-U = Rangtest, der prüft, ob ein Unterschied echt ist), nicht nur per Augenmaß.
- „**Verfügbarkeit = eigene Achse**" — Zuverlässigkeit wird getrennt von der Latenz berichtet (s. Folie 3).

Mögliche Nachfrage: „Warum Median der Slot-Mediane statt einfach alles zusammen?" → „Sonst gewichten Tageszeiten mit mehr erfolgreichen Aufrufen die Zahl stärker; der zweistufige Median gibt jeder Tageszeit gleiches Gewicht."

Folie 15 — Abschnitt 3: Analyse (Trenner)

Trenner. Markiert klar: „**56 von 56 Slots · A4-Median (CI folgt)**" — alle folgenden Zahlen sind die A4-Auswertung über alle 56 Slots (Vollkampagne, abgeschlossen); die Konfidenzintervalle stehen noch aus. Unterpunkte: Kernbefund, Interpretation, Limitationen.

Folie 16 — Kernbefund · gleicher Edge, $\sim 7,3\times$ ■■ (das Herzstück)

Das Balkendiagramm: LLM-ttft (Zeit bis zum ersten Antwort-Token), connect-inklusiv, 56/56 Slots, A4: **Groq ~ 67 ms · Mistral ~ 279 ms · OpenAI ~ 487 ms** (exakt 66,9 / 278,9 / 486,6).

- „**Alle 3 LLM @ Cloudflare FRA**" — die drei Sprachmodell-Anbieter enden alle am selben Cloudflare-Knoten in Frankfurt (Folie 11).
- „**100 % des Traffics belegt (2 CF-IPs, AS13335, ~ 1 ms)**" — das gilt nicht nur stichprobenartig: Jeder Anbieter wird per DNS über **zwei** Cloudflare-IPs ($\sim 50/50$) bedient, **beide** AS13335 mit ~ 1 ms — für den **gesamten** LLM-Verkehr gemessen, nicht unterstellt.
- „**ttft streut $\sim 7,3\times$ (Groq→Mistral→OpenAI)**" — trotz identischem Netz ist OpenAI $\sim 7,3$ -mal langsamer als Groq (486,6 / 66,9). Die Reihenfolge ist Groq < Mistral < OpenAI.
- „**Pro IP stabil (kein Edge-Effekt)**" — egal welche der beiden Cloudflare-IPs getroffen wird, das Ergebnis bleibt gleich. Der „Edge-Wechsel" hat **null** Effekt → die Spreizung kommt nicht vom Netz-Rand.
- „**Region-Reihenfolge gedreht: Mistral (EU) langsamer als Groq (US)**" — die **deklarierte** Geografie ist sogar **umgekehrt**: der EU-Anbieter Mistral ist langsamer als der US-Anbieter Groq. Wäre Netznähe die Erklärung, müsste es andersherum sein.

- „**Gleiches Netz, ~7,3× → Netznähe erklärt das nicht**“ — die Schlussfolgerung: identische Netzdistanz, riesiger Latenzunterschied → die Ursache **kann nicht** die Netznähe sein.
- **Fußnote: „Ein Vantage Point (FRA) · Edge-Befund FRA-spezifisch — die Kernlogik (gleiches Netz → Spreizung bleibt) unberührt · Bootstrap-CI folgen.“** — wiederholt ehrlich die Single-VP-Grenze, betont aber: die **Logik** des Befundes hängt nicht am Standort.

Ein Satz (Kernsatz der ganzen Arbeit): „Drei LLM am selben Cloudflare-Edge in Frankfurt, ~1 ms für alle — und trotzdem ~7,3× Unterschied, pro IP stabil, Geografie sogar invertiert. Gleiches Netz kann das nicht erklären.“

Mögliche Nachfrage: „Sind 7,3× nicht zufällig?“ → „Die Punktschätzer sind über alle 56 Slots robust (A4-Median), die Reihenfolge ist stabil; die Bootstrap-Konfidenzintervalle rechne ich für die finale Fassung. Der frühere 16-Slot-Zwischenstand zeigte 6,5× — die Größenordnung ist also stabil, mit mehr Daten sogar größer.“

Folie 17 — Interpretation · der Kompass

Die rote Box: „Verteidigbar ist die negative Aussage: ‚Netznähe erklärt die Latenzspreizung nicht.‘“ — der wasserdichte Kern. Alles Weitere ist Interpretation mit Vorbehalt.

- „**Schritt zu Backend trägt Modellgrößen-Confound**“ — wenn ich sage „dann liegt's am Backend“, gibt es einen **Störfaktor (Confound)**: Der schnellste Anbieter (Grog) hat **gleichzeitig** das kleinste Modell **und** spezielle Hardware. Ich kann die beiden Effekte nicht sauber trennen.
- „**Grog = kleinstes Modell + LPU**“ — konkretes Beispiel des Confounds: klein **und** Spezial-Chip.
- „→ **Backend (Hardware + Modell) statt Geografie**“ — deshalb die vorsichtige Formulierung: Der Unterschied liegt im **Backend** (HW + Modell zusammen), **nicht** in der Geografie — aber ich behaupte nicht „nur die Rechenleistung“.
- „**2. Beleg TTS: Azure ~94 ms = schnellstes***“ — bei der Sprachausgabe ist Azure am schnellsten (ttfa ~94 ms). Das Sternchen verweist auf die Verfügbarkeits-Fußnote.
- „**OpenAI-TTS auch @ CF-FRA → ~942 ms ttfa = fast reines Backend (connect ~1 ms)**“ — **zweiter identical-edge-Beleg**: OpenAI-TTS endet am **gleichen** Cloudflare-Edge (~1 ms), ist aber ~942 ms langsam. Da der Verbindungsaufbau nur ~1 ms ist, ist fast die gesamte Zeit **reines Backend**. Stützt den Kernbefund ein zweites Mal.
- **Fußnote:** „*,* nur erfolgreiche Calls; OpenAI-TTS ~3,1 % Timeout-Ausfälle (Vollkampagne). ‚Schnellstes‘ gilt ggü. Deepgram. Azure-TTS-Median ~94 ms (echtes Backend-Tail in p95/p99).“ — drei Ehrlichkeiten: Latenz nur über erfolgreiche Aufrufe (OpenAI-TTS fällt zu ~3,1 % aus); „Azure am schnellsten“ gilt sauber gegenüber Deepgram; und Azures **Median** ist niedrig, aber es gibt einen langsamen **Schwanz** (p95/p99), den man nicht verschweigt. (p95/p99 = die langsamsten 5 % bzw. 1 % der Aufrufe.)

Interpretations-Kompass (wichtig, nicht überziehen): „Wasserdicht ist nur die negative Aussage. Den Schritt zu ‚Backend‘ formuliere ich offen, weil Modellgröße und Hardware verquickt sind — deshalb steht auch kein ‚Engine‘ im Titel.“

Folie 18 — Ehrliche Limitationen

Diese Folie zeigt Reife: Du benennst die Grenzen selbst, bevor der Prüfer sie findet.

- „**Ein Vantage Point (FRA): Edge FRA-spezifisch; Kernbefund robust**“ — nur ein Standort; RTT-/Edge-Zahlen gelten für Frankfurt. Der Kernbefund bleibt robust (konstante Netzdistanz).
- „**STT stützt den Kernbefund nicht — ttfp durch Pacing-Untergrenze dominiert (~0,8 s)**“ — **ehrlich**: Die Spracherkennung trägt den Kernbefund **nicht**. Weil das Audio im Echtzeit-Tempo gestreamt wird, kommt das erste Wort bei allen Anbietern erst nach ~0,8 s — dieser gemeinsame

„Boden“ überdeckt die Engine-Unterschiede.

- „**Frühere ‚Endpointing-Inversion‘: Auswertungs-Artefakt** → **gestrichen**“ — eine alte These (Azure-STT sei durch Stille-Erkennung langsam) war ein **Artefakt** der alten Auswertung (Audio auf einen Schlag statt im Takt) → ehrlich gestrichen.
- „**STT = Kontrast-/Verfügbarkeitsfall**“ — STT bleibt nützlich als Gegenbeispiel und für die Verfügbarkeits-Achse, nicht als Engine-Beleg.
- „**Vollkampagne, abgeschlossen (56/56); Bootstrap-CI noch ausstehend**“ — Datenstand ehrlich: alle 56 von 56 Slots, die Konfidenzintervalle fehlen noch.
- „**Diurnal = Snapshot (keine Tageszeit-Inferenz)**“ — über Tageszeit-Effekte (*diurnal* = tageszeitabhängig) treffe ich noch **keine** belastbare Aussage; dafür ist das Design noch nicht orthogonal genug.
- „**Nur der Connect-Timer ist paket-geeicht**“ — Wiederholung der ehrlichen Eich-Reichweite (Folie 12).
- „**Gesamtdauer ist längenabhängig** → **ttft/ttfa als Primärmetrik**“ — die **Gesamtdauer** hängt davon ab, *wie viel* ein Modell ausgibt; deshalb ist die **Erst-Token-Zeit** (ttft/ttfa) die faire Hauptmetrik (mengen-unabhängig).

Ein Satz: „Die ehrlichste Limitation ist, dass STT den Kernbefund nicht trägt — das sage ich ausdrücklich; der Befund ruht auf der LLM-Achse und einem zweiten TTS-Beleg.“

Folie 19 — Abschnitt 4: Abschluss (Trenner)

Trenner. Stand · nächste Schritte · Diskussion.

Folie 20 — Stand & nächste Schritte

- „**Messinfrastruktur steht und läuft stabil**“ — das Mess-System ist fertig und zuverlässig.
- „**Methodik automatisiert mehrfach gegengeprüft**“ — ich habe die Methodik wiederholt (automatisiert) gegen die Daten geprüft. (*Bewusst neutral formuliert — kein internes Audit-Vokabular vor dem Prüfer.*)
- „→ **offene Punkte betreffen nur Doku/Reporting, nicht die Daten**“ — die gefundenen Korrekturen waren Darstellungs-/Dokumentationsfragen, **keine** Mess- oder Datenfehler.
- „**Vollkampagne abgeschlossen (56/56) → Statistik (Bootstrap-CI + Inferenz)**“ — nach Kampagnenende kommt die volle Statistik (Konfidenzintervalle, Tests).
- „**Ausarbeitung, Ziel ~August 2026**“ — geplanter Fertigstellungszeitraum.
- „**Zweitgutachten: Prof. Färber zugesagt (Exposé, Gespräch Juli)**“ — Stand der Zweitprüfer-Frage.
- **Diskussions-Box:** „**Welche Titelrichtung finden Sie am sinnvollsten?**“ — bewusster Gesprächs-Aufhänger für den Prof (offene Frage / deskriptiv / methodenfokussiert).

Hinweis: Wenn der Prof beim Wort „gegengeprüft/Audit“ nachfragt: Es ist eine **eigene, automatisierte Methodik-Selbstprüfung**, **kein** externes Gutachten — so auch sagen.

Folie 21 — Backup · Klassifikator & Per-IP-Belege

(Backup = nur zeigen, wenn nachgefragt wird.)

- „**Edge nur, wenn alle drei zutreffen**“ — Wiederholung des Klassifikators (Folie 11), hier mit Belegen.
- **(a) einstellige ms FRA :443 / (b) IP in AS13335 (Team-Cymru) / (c) Traceroute endet im CDN-AS** — die drei Bedingungen. *Team-Cymru* ist der Dienst, mit dem ich nachschlage, zu welchem Netzbetreiber (AS) eine IP gehört.

- „**Belegt über alle getroffenen IPs**“ — nicht nur an einer Stichprobe, sondern über **jede** im Betrieb getroffene IP.
- „**Deepgram: zwei US-Carrier (AS6461 / AS174), ~101–140 ms**“ — Deepgram verteilt per DNS auf zwei US-Netzbetreiber (Zayo/Cogent) mit leicht unterschiedlicher RTT — erklärt, warum „ein Host“ eine Spanne zeigt.
- „**Rev.ai AS16509 ~140 ms · Azure AS8075 ~11 ms**“ — Rev.ai bei Amazon (US), Azure bei Microsoft (echtes EU-RZ).
- „**Bei Edge ist connect $\approx N \times \text{RTT}$ zwangsläufig $\rightarrow$ kein eigener Beleg**“ — Feinheit für die Cross-Layer-Brücke: Bei Edge-Anbietern ist „Verbindungsaufbau $\approx$ Vielfaches der RTT“ trivial wahr (gleicher Knoten) und zählt deshalb **nicht** als zusätzlicher Beweis; nur host-terminierte Anbieter stützen die Brücke.

Folie 22 — Backup · Aggregation & Inferenz (Detail)

- „**p50/p90 slot-aufgelöst · p95 nur gepoolt + CI · p99 nicht**“ — $p50/p90/p95/p99$ = Perzentile (der Wert, unter dem 50/90/95/99 % der Messungen liegen). p50/p90 sind je Slot belastbar; p95 nur über alle Slots zusammen; p99 behaupte ich gar nicht (zu wenige Daten pro Slot).
- „**Faustregel: $n \cdot (1-q) \geq 5-10$** “ — eine Perzentil-Schätzung ist erst stabil, wenn genug extreme Werte vorliegen; diese Faustregel sagt, ab wann.
- „**Stark rechtsschief $\rightarrow$ Bootstrap statt t**“ — die Latenz-Verteilungen haben einen langen Schwanz nach oben (*rechtsschief*); deshalb Bootstrap statt des klassischen t-Konfidenzintervalls (das Symmetrie annimmt).
- „**E2E: Monte-Carlo-Faltung der 3 Phasen**“ — die Gesamt-Pipeline-Latenz berechne ich, indem ich aus den drei Phasen-Verteilungen viele Male zufällig ziehe und addiere (*Monte-Carlo*), statt nur drei Mediane zu summieren.
- „**Joint-Completion / Pareto-Front (Latenz vs. Zuverlässigkeit)**“ — ich trage Geschwindigkeit **gegen** Zuverlässigkeit auf; ein „Gewinner“ wird erst nach Abwägung beider benannt (*Pareto-Front* = die Menge der besten Kompromisse).
- „**IP-Quelle: aufgelöste Connect-IP — Fehlschläge nie über Null-Werte filtern**“ — technische Disziplin: Für IP-/Region-Analysen die richtige IP-Quelle nehmen und gescheiterte Aufrufe nicht versehentlich herausfiltern (sonst verschwindet gerade die Verfügbarkeits-Information).

Folie 23 — Backup · Fehlerbehandlung & Verfügbarkeit

- „**Erfolg = inhaltlich gültiger Output (nicht nur Verbindung)**“ — ein Aufruf gilt erst als Erfolg, wenn **brauchbare** Ausgabe kam (echtes Transkript / echte Token / echtes Audio), nicht schon, wenn die Verbindung stand. Sonst zählen leere Antworten fälschlich als „schnell“.
- „**Timeout-Filter: Teilstring ‚ReadTimeout‘ klassifizieren**“ — Zeitüberschreitungen erkenne ich am Fehler-**Text** (Teilstring „ReadTimeout“), weil es kein sauberes Fehler-Kategorie-Feld gibt.
- „**nicht nach error==‘timeout‘ (verfehlt fast alle echten Timeouts)**“ — ein naiver Filter auf das Wort „timeout“ würde fast alle echten Zeitüberschreitungen übersehen (sie heißen „ReadTimeout“). Das dokumentiere ich, damit die Verfügbarkeitszahlen reproduzierbar sind.
- „**OpenAI-TTS ~3,1 % ReadTimeouts (173/5600, Vollkampagne)**“ — konkret: OpenAI-TTS scheitert in der Vollkampagne zu ~3,1 % an Zeitüberschreitungen.
- „ **$\rightarrow$ Latenz nur erfolgreiche Calls + Asterisk**“ — deshalb wird die Latenz nur über erfolgreiche Aufrufe gebildet und „schnellstes TTS“ immer mit Ausfall-Hinweis (Sternchen) versehen.

Zur Tabelle (Timeouts/Erfolgskriterien): Connect-Timeout 10 s, Response-Timeout 30 s (alle Kategorien); „Erfolg“ = STT nicht-leeres Final-Transkript / LLM ≥ 1 Wort und ≥ 3 Antwort-Häppchen / TTS

Spickzettel — die Zahlen, die sitzen müssen (final, 56/56 Slots, A4)

Größe	Wert
LLM-ttft Groq / Mistral / OpenAI	67 / 279 / 487 ms (66,9 / 278,9 / 486,6)
Faktor langsamster/schnellster	~7,3× (gepoolt 8,3×); Mistral/Groq ~4,2×
LLM-RTT (alle)	~1 ms (Cloudflare FRA, AS13335)
Azure-TTS / OpenAI-TTS ttfa	~94 ms / ~942 ms (OpenAI-TTS connect ~1 ms)
OpenAI-TTS Fail-Rate	~3,1 % (173/5600)
STT-ttfa Azure / Deepgram / Rev.ai	1045 / 1045 / 1494 ms
Layer-2-Eichung (Connect-Timer)	±~0,1 ms (Azure 11, Deepgram 139 ms)
Kampagne	7 T × 8 Slots × n=100 = 56 Slots / 50.400 Calls ; Vollkampagne abgeschlossen 56/56

Drei Sätze, die immer passen:

1. „Gleicher Edge (~1 ms), trotzdem ~7,3× — die Netznähe erklärt die Spreizung nicht.“ (Kernbefund, negativ & wasserdicht)
2. „Den Schritt zu ‚Backend‘ formuliere ich offen, weil Modellgröße und Hardware verquickt sind.“ (Kompass)
3. „Das sind A4-Mediane über alle 56 Slots; die Bootstrap-CI stehen noch aus, die Ordnung ist stabil.“ (Ehrlichkeit)